Introduction to Big Data

Jan 2025 Term – Graded Assignment 7

Name :- Satvik Chandrakar

Roll no :- 21f1000344

Task :- Create an input data file that has at least 1000 rows and place it in a GCS bucket.

Write a Producer that reads from that file, breaks the data into batches of 10 records, and writes to Kafka, such that each batch is separated by a sleep time of 10 seconds from the previous batch. The Producer stop emitting once 1000 records are written.

Write a Spark Streaming consumer that reads from the same Kafka topic every 5 seconds and emits count of rows seen in the last 10 seconds

Launch both the Producer and Spark Streaming in 2 separate windows such that both are running simultaneously.

The runs can stop when all 1000 rows are consumed.

My Approach :-

- Step 1 :- Opened the Google Cloud Shell
- Step 2 :- Run the following commands to authenticate:
    - *gcloud auth login*
    - *gcloud config set eminent-crane-448810-s3*
- Step 3 :- Created create_vm.sh to deploy a virtual machine.
    - *nano create_vm.sh*
    - wrote the commands
    - saved it and exited the text editor
      CTRL + X -> Y -> ENTER
    - then run the following command:
      chmod +x create_vm.sh
    - then moved to parent directory using command *cd –*
    - executed the script
      ./create_vm.sh

Cloud Shell Editor

(eminent-crane-448810-s3)

```
  GNU nano 7.2                                                    create_vm.sh
#!/bin/bash

# Variables
PROJECT_ID="eminent-crane-448810-s3"
ZONE="us-central1-a"
VM_NAME="kafka-spark-vm-2"
MACHINE_TYPE="c4-standard-4"
IMAGE_FAMILY="debian-11"
IMAGE_PROJECT="debian-cloud"

# Step 1: Create VM instance
gcloud compute instances create $VM_NAME \
    --project=$PROJECT_ID \
    --zone=$ZONE \
    --machine-type=$MACHINE_TYPE \
    --image-family=$IMAGE_FAMILY \
    --image-project=$IMAGE_PROJECT \
    --boot-disk-size=200GB \
    --scopes=storage-full,cloud-platform \
    --tags=kafka-server,spark-server

# Step 2: Open required ports
gcloud compute firewall-rules create kafka-port --allow tcp:9092 --target-tags kafka-server --quiet
gcloud compute firewall-rules create spark-ui-port --allow tcp:4040 --target-tags spark-server --quiet
```

- Step 4 :- Created ssh_vm.sh to SSH into the virtual machine.
    - *nano ssh_vm.sh*
    - wrote the commands
    - saved it and exited the text editor
        CTRL + X -> Y -> ENTER
    - then run the following command:
        chmod +x ssh_vm.sh
    - then moved to parent directory using command *cd –*
    - executed the script
        ./ssh_vm.sh

Cloud Shell Editor

(eminent-crane-448810-s3)

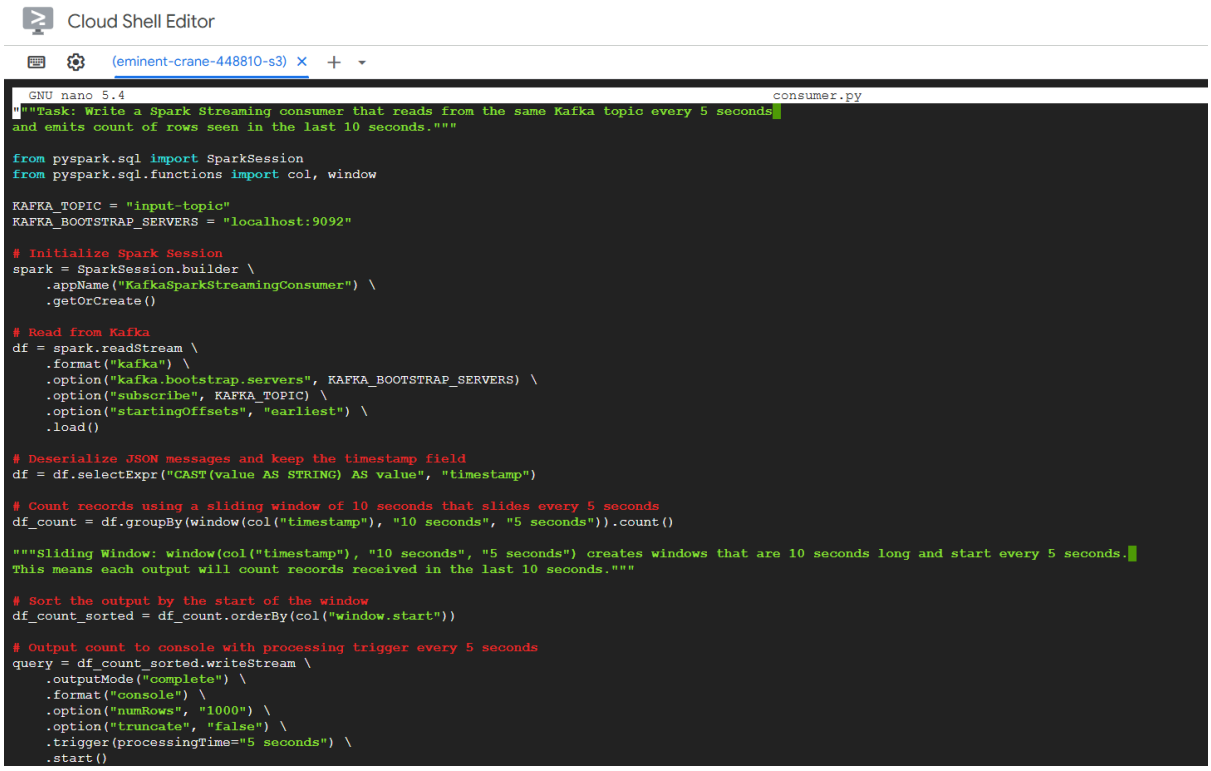
```
  GNU nano 7.2                                                      ssh_vm.sh
#!/bin/bash

# Variables
ZONE="us-central1-a"
VM_NAME="kafka-spark-vm-2"

# SSH into the VM
gcloud compute ssh $VM_NAME --zone=$ZONE
```

```
chandrakarsatvik@cloudshell:~ (eminent-crane-448810-s3)$ ./ssh_vm.sh
Enter passphrase for key '/home/chandrakarsatvik/.ssh/google_compute_engine':
Linux kafka-spark-vm-2 5.10.0-33-cloud-amd64 #1 SMP Debian 5.10.226-1 (2024-10-03) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Mar  9 14:04:58 2025 from 34.143.216.166
chandrakarsatvik@kafka-spark-vm-2:~$
```

Note :- From step 5 onward all the steps were executed inside the VM

- Step 5 :- Created consumer.py. It reads from the Kafka topic every 5 seconds and emits count of rows seen in the last 10 seconds.
    - *nano consumer.py*
    - Added the python code to consumer.py
    - Saved it and exited the text editor

CTRL + X -> Y -> ENTER



- Step 6 :- Created producer.py. It reads from that file, breaks the data into batches of 10 records, and writes to Kafka, such that each batch is separated by a sleep time of 10 seconds from the previous batch. It stops emitting once 1000 records are written.
    - *nano producer.py*
    - Added the python code to producer.py
    - Saved it and exited the text editor

CTRL + X -> Y -> ENTER

(eminent-crane-448810-s3)

```
 GNU nano 5.4                                                                producer.py
"""Task: Write a Producer that reads from that file, breaks the data into batches of 10 records, and writes to Kafka,
such that each batch is separated by a sleep time of 10 seconds from the previous batch.
The Producer can stop emitting once 1000 records are written."""

import time
from google.cloud import storage
from kafka import KafkaProducer

BUCKET_NAME = "satvik-storage-bucket"
FILE_NAME = "input_data.csv"
KAFKA_TOPIC = "input-topic"
KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"

producer = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS, value_serializer=lambda x: x.encode('utf-8'))

# Download file from GCS
client = storage.Client()
bucket = client.bucket(BUCKET_NAME)
blob = bucket.blob(FILE_NAME)
data = blob.download_as_text().split("\n")

batch_size = 10
for i in range(0, len(data), batch_size):
    batch = data[i:i + batch_size]
    if not batch or batch[0] == "":
        continue

    for record in batch:
        producer.send(KAFKA_TOPIC, record)

    print(f"Sent batch {i//batch_size + 1}")
    time.sleep(10)  # 10 seconds delay

print("Producer finished sending messages.")
producer.close()
```

- Step 7 :- Created create_input_data.py. It creates an input data file that has 1000 rows.
  - *nano create_input_data.py*
  - Added the python code to create_input_data.py
  - Saved it and exited the text editor

    CTRL + X -> Y -> ENTER

(eminent-crane-448810-s3)

```
 GNU nano 5.4                                                              create_input_data.py
import pandas as pd
import random

# Create sample data
data = [{"id": i, "value": random.randint(1, 100)} for i in range(1000)]
df = pd.DataFrame(data)

# Save as CSV
df.to_csv("input_data.csv", index=False)
```

- Step 8 :- Created install_dependencies.sh in the script directory and wrote commands to download java, kafka, spark and other required packages.
  - *cd scripts*
  - *nano install_dependencies.sh*
  - wrote the commands
  - saved it and exited the text editor

    CTRL + X -> Y -> ENTER
  - then run the following command:

    chmod +x install_dependencies.sh
  - then moved to parent directory using command *cd –*
  - executed the script

    ./scripts/install_dependencies.sh

(eminent-crane-448810-s3)

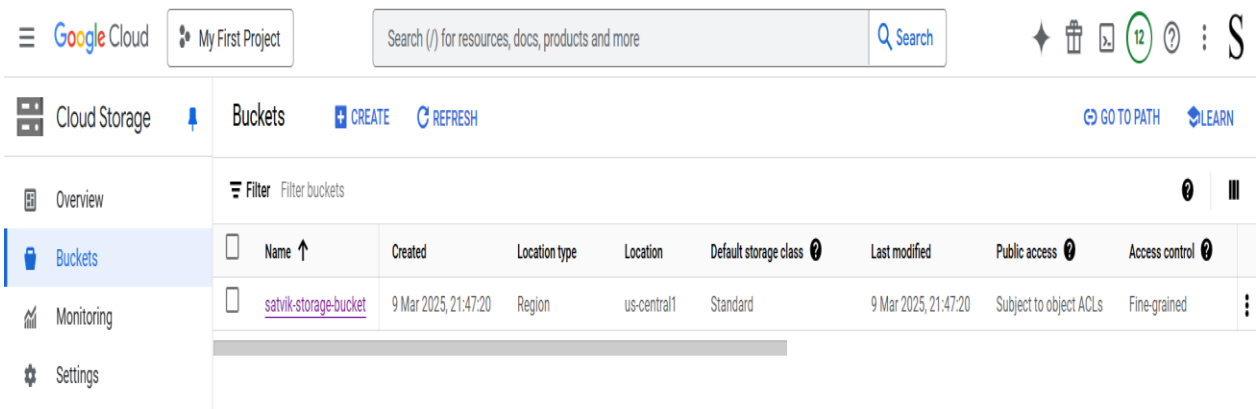
```
  GNU nano 5.4                                                    install_dependencies.sh
# To be executed inside VM
# Install Java
sudo apt update
sudo apt install default-jdk -y
java -version
# Download & Extract Kafka
wget https://downloads.apache.org/kafka/3.7.2/kafka_2.13-3.7.2.tgz
tar -xvzf kafka_2.13-3.7.2.tgz
mv kafka_2.13-3.7.2 kafka
# Download & Extract Spark
wget https://downloads.apache.org/spark/spark-3.5.5/spark-3.5.5-bin-hadoop3.tgz
tar -xvzf spark-3.5.5-bin-hadoop3.tgz
mv spark-3.5.5-bin-hadoop3 spark
echo 'Kafka & Spark setup completed!'
# Packages
sudo apt update && sudo apt install -y google-cloud-sdk python3 python3-pip scala
pip3 install google-cloud-storage kafka-python pyspark pandas
```

- Step 9 :- Created create_gcs_bucket.sh in scripts directory to deploy Google Cloud Storage bucket with the name satvik-storage-bucket and rest of the configurations in the default setting and executed the script.
    - *cd scripts*
    - *nano create_gcs_bucket.sh*
    - wrote the commands
    - saved it and exited the text editor
        CTRL + X -> Y -> ENTER
    - then run the following command:
        chmod +x create_gcs_bucket.sh
    - then moved to parent directory using command *cd –*
    - executed the script
        ./scripts/create_gcs_bucket.sh

(eminent-crane-448810-s3)

```
  GNU nano 5.4                                                    create_gcs_bucket.sh
gcloud storage buckets create gs://satvik-storage-bucket --location=us-central1
```

```
chandrakarsatvik@kafka-spark-vm-2:~$ ./scripts/create_gcs_bucket.sh
Creating gs://satvik-storage-bucket/...
chandrakarsatvik@kafka-spark-vm-2:~$
```



- Step 10 :- Executed the create_input_data.py and input_data.csv was successfully created.

```
chandrakarsatvik@kafka-spark-vm-2:~$ ls
consumer.py  create_input_data.py  kafka  kafka_2.13-3.7.2.tgz  producer.py  scripts  spark  spark-3.5.5-bin-hadoop3.tgz
chandrakarsatvik@kafka-spark-vm-2:~$ python3 create_input_data.py
chandrakarsatvik@kafka-spark-vm-2:~$ ls
consumer.py  create_input_data.py  input_data.csv  kafka  kafka_2.13-3.7.2.tgz  producer.py  scripts  spark  spark-3.5.5-bin-hadoop3.tgz
chandrakarsatvik@kafka-spark-vm-2:~$
```

```
chandrakarsatvik@kafka-spark-vm-2:~$ cat input_data.csv
id,value
0,52
1,87
2,90
3,13
4,6
```

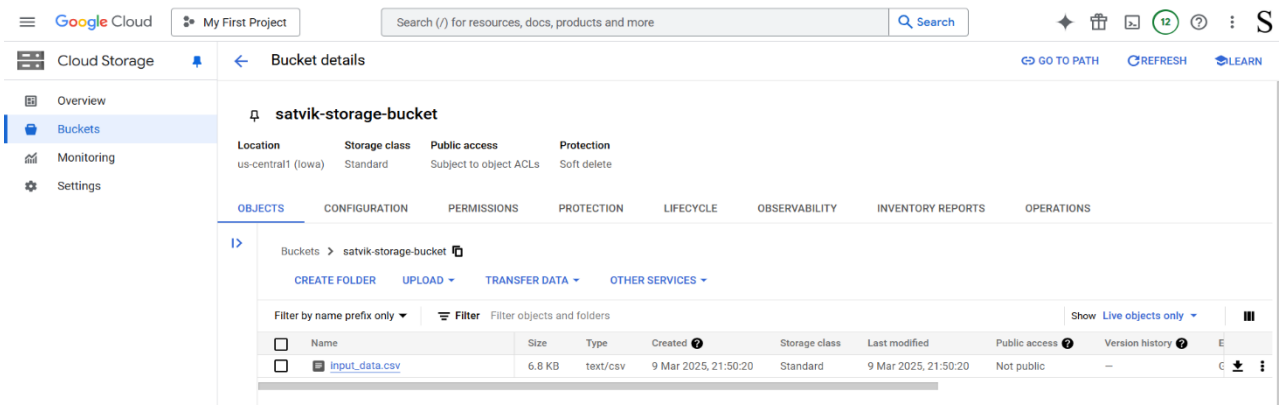
- Step 11 :- Created upload_files_gcs_bucket.sh in scripts directory to upload the input_data.csv to GCS bucket.
    - *cd scripts*
    - *nano upload_files_gcs_bucket.sh*
    - wrote the command
    - saved it and exited the text editor

      CTRL + X -> Y -> ENTER
    - then run the following command:

      chmod +x upload_files_gcs_bucket.sh
    - then moved to parent directory using command *cd –*
    - executed the script

      ./scripts/upload_files_gcs_bucket.sh

```
chandrakarsatvik@kafka-spark-vm-2:~$ ./scripts/upload_files_gcs_bucket.sh
Copying file:///home/chandrakarsatvik/input_data.csv [Content-Type=text/csv]...
/ [1 files][  6.6 KiB/  6.6 KiB]
Operation completed over 1 objects/6.6 KiB.
chandrakarsatvik@kafka-spark-vm-2:~$
```

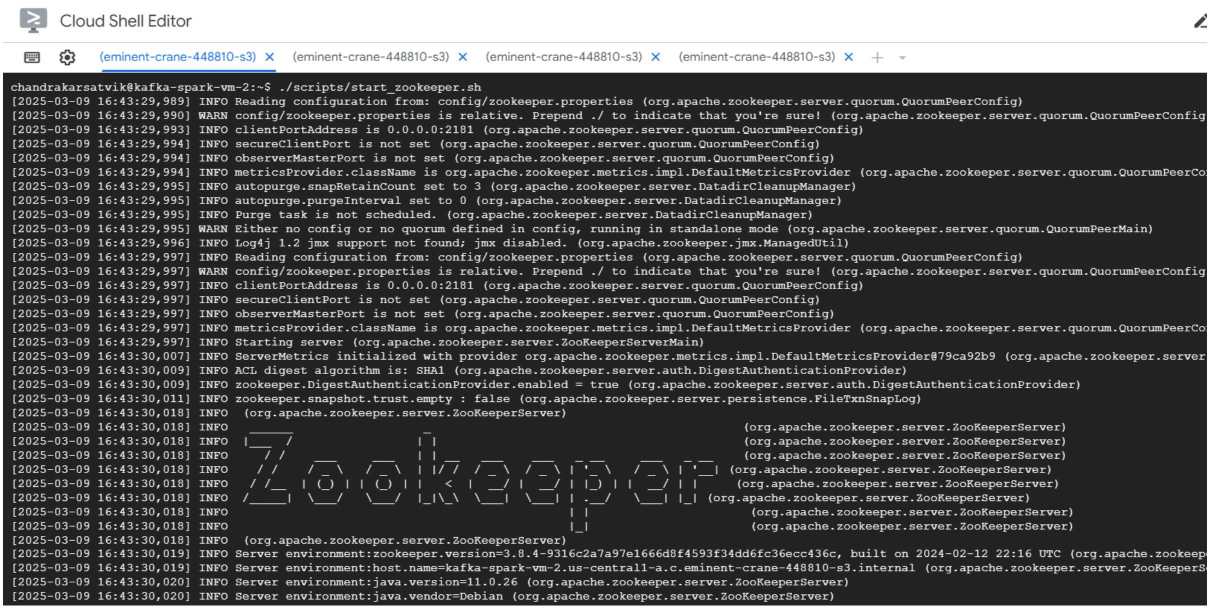


- Step 12 :- Created start_zookeeper.sh in scripts directory to start the zookeeper in the 1st terminal.
    - *cd scripts*
    - *nano start_zookeeper.sh*
    - wrote the command
    - saved it and exited the text editor

      CTRL + X -> Y -> ENTER
    - then run the following command:

      chmod +x start_zookeeper.sh
    - then moved to parent directory using command *cd –*
    - executed the script

      ./scripts/start_zookeeper.sh



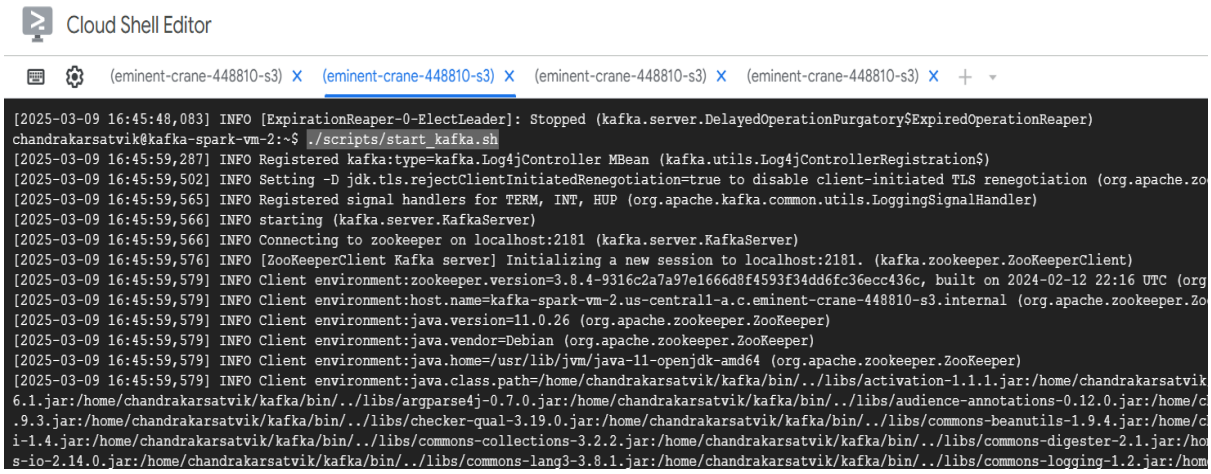
- Step 13 :- Created start_kafka.sh in scripts directory to start the kafka in the 2nd terminal.
    - *cd scripts*
    - *nano start_kafka.sh*
    - wrote the command
    - saved it and exited the text editor

      CTRL + X -> Y -> ENTER
    - then run the following command:

      chmod +x start_kafka.sh
    - then moved to parent directory using command *cd –*

- executed the script

./scripts/start_kafka.sh

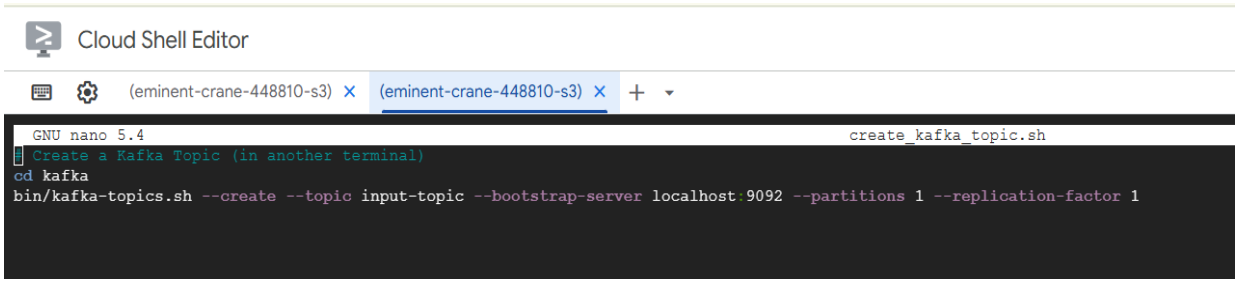


- Step 14 :- Created create_kafka_topic.sh in scripts directory to create the kafka topic on which producer will write. This was executed in the 3rd terminal.
  - *cd scripts*
  - *nano create_kafka_topic.sh*
  - wrote the command
  - saved it and exited the text editor

    CTRL + X -> Y -> ENTER
  - then run the following command:

    chmod +x create_kafka_topic.sh
  - then moved to parent directory using command *cd –*
  - executed the script

    ./scripts/create_kafka_topic.sh

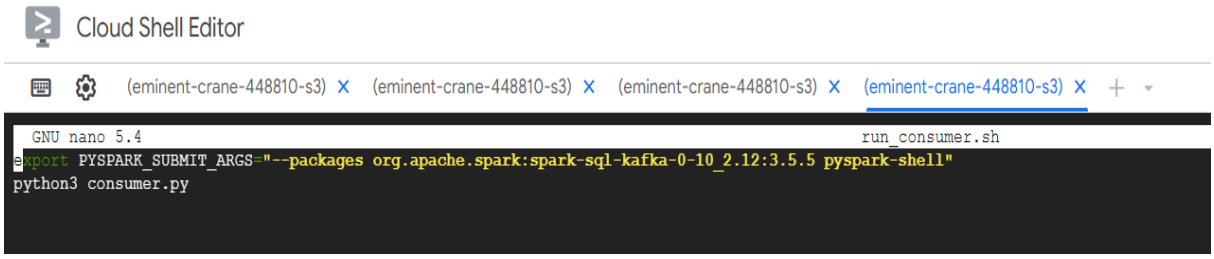


```
chandrakarsatvik@kafka-spark-vm-2:~$ ./scripts/create_kafka_topic.sh
Created topic input-topic.
chandrakarsatvik@kafka-spark-vm-2:~$ cd kafka
chandrakarsatvik@kafka-spark-vm-2:~/kafka$ bin/kafka-topics.sh --list --bootstrap-server localhost:9092
input-topic
chandrakarsatvik@kafka-spark-vm-2:~/kafka$ cd -
/home/chandrakarsatvik
chandrakarsatvik@kafka-spark-vm-2:~$
```

- Step 15 :- Created run_consumer.sh in scripts directory to execute the consumer.py.
    - *cd scripts*
    - *nano run_consumer.sh*
    - wrote the command
    - saved it and exited the text editor

      CTRL + X -> Y -> ENTER
    - then run the following command:

      chmod +x run_consumer.sh
    - then moved to parent directory using command *cd –*



- Step 16 :- In the 3rd terminal executed the producer.py and it is successfully doing its job.

- Step 17 :- In the 4th terminal executed the run_consumer.sh script and it is successfully doing its job.

Cloud Shell Editor

(eminent-crane-448810-s3) × (eminent-crane-448810-s3) × (eminent-crane-448810-s3) × (eminent-crane-448810-s3) ×

```
        at org.apache.spark.sql.execution.SparkPlan.executeQuery(SparkPlan.scala:243)
        at org.apache.spark.sql.execution.SparkPlan.execute(SparkPlan.scala:191)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.writeWithV2(WriteToDataSourceV2Exec.scala:366)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.writeWithV2$(WriteToDataSourceV2Exec.scala:364)
chandrakarsatvik@kafka-spark-vm-2:~$ ./scripts/run_consumer.sh
:: loading settings :: url = jar:file:/home/chandrakarsatvik/.local/lib/python3.9/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /home/chandrakarsatvik/.ivy2/cache
The jars for the packages stored in: /home/chandrakarsatvik/.ivy2/jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-0f1e8e17-0a05-4f73-a62e-baf29d971415;1.0
        confs: [default]
        found org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.5 in central
        found org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.5 in central
        found org.apache.kafka#kafka-clients;3.4.1 in central
        found org.lz4#lz4-java;1.8.0 in central
        found org.xerial.snappy#snappy-java;1.1.10.5 in central
        found org.slf4j#slf4j-api;2.0.7 in central
        found org.apache.hadoop#hadoop-client-runtime;3.3.4 in central
        found org.apache.hadoop#hadoop-client-api;3.3.4 in central
        found commons-logging#commons-logging;1.1.3 in central
        found com.google.code.findbugs#jsr305;3.0.0 in central
        found org.apache.commons#commons-pool2;2.11.1 in central
:: resolution report :: resolve 351ms :: artifacts dl 12ms
        :: modules in use:
        com.google.code.findbugs#jsr305;3.0.0 from central in [default]
        commons-logging#commons-logging;1.1.3 from central in [default]
        org.apache.commons#commons-pool2;2.11.1 from central in [default]
        org.apache.hadoop#hadoop-client-api;3.3.4 from central in [default]
        org.apache.hadoop#hadoop-client-runtime;3.3.4 from central in [default]
        org.apache.kafka#kafka-clients;3.4.1 from central in [default]
        org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.5 from central in [default]
        org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.5 from central in [default]
        org.lz4#lz4-java;1.8.0 from central in [default]
        org.slf4j#slf4j-api;2.0.7 from central in [default]
        org.xerial.snappy#snappy-java;1.1.10.5 from central in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   11  |   0   |   0   |   0   ||   11  |   0   |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-0f1e8e17-0a05-4f73-a62e-baf29d971415
        confs: [default]
        0 artifacts copied, 11 already retrieved (0kB/6ms)
25/03/09 17:01:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

Cloud Shell Editor

(eminent-crane-448810-s3) × (eminent-crane-448810-s3) × (eminent-crane-448810-s3) × (eminent-crane-448810-s3) ×

```
        |      default     |   11  |                       11  |   0   |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-0f1e8e17-0a05-4f73-a62e-baf29d971415
        confs: [default]
        0 artifacts copied, 11 already retrieved (0kB/6ms)
25/03/09 17:01:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/03/09 17:01:17 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when the query didn't fail: /tmp/temporary-67e371f8-8ded-
t's required to delete it under any circumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting temp checkpoi
25/03/09 17:01:17 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
25/03/09 17:01:17 WARN AdminClientConfig: These configurations '[key.deserializer, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were
t.
-------------------------------------------
Batch: 0
-------------------------------------------
+------------------------------------------+-----+
|window                                    |count|
+------------------------------------------+-----+
|{2025-03-09 16:51:20, 2025-03-09 16:51:30}|10   |
|{2025-03-09 16:51:25, 2025-03-09 16:51:35}|10   |
|{2025-03-09 16:51:30, 2025-03-09 16:51:40}|10   |
|{2025-03-09 16:51:35, 2025-03-09 16:51:45}|10   |
|{2025-03-09 16:51:40, 2025-03-09 16:51:50}|10   |
|{2025-03-09 16:51:45, 2025-03-09 16:51:55}|10   |
|{2025-03-09 16:52:05, 2025-03-09 16:52:15}|10   |
|{2025-03-09 16:52:10, 2025-03-09 16:52:20}|10   |
|{2025-03-09 16:52:15, 2025-03-09 16:52:25}|10   |
|{2025-03-09 16:52:20, 2025-03-09 16:52:30}|10   |
|{2025-03-09 16:52:25, 2025-03-09 16:52:35}|10   |
|{2025-03-09 16:52:30, 2025-03-09 16:52:40}|10   |
|{2025-03-09 16:52:35, 2025-03-09 16:52:45}|10   |
|{2025-03-09 16:52:40, 2025-03-09 16:52:50}|10   |
|{2025-03-09 16:52:45, 2025-03-09 16:52:55}|10   |
|{2025-03-09 16:52:50, 2025-03-09 16:53:00}|10   |
|{2025-03-09 16:52:55, 2025-03-09 16:53:05}|10   |
|{2025-03-09 16:53:00, 2025-03-09 16:53:10}|10   |
|{2025-03-09 16:53:05, 2025-03-09 16:53:15}|10   |
|{2025-03-09 16:53:10, 2025-03-09 16:53:20}|10   |
|{2025-03-09 16:53:15, 2025-03-09 16:53:25}|10   |
|{2025-03-09 16:53:20, 2025-03-09 16:53:30}|10   |
|{2025-03-09 16:53:25, 2025-03-09 16:53:35}|10   |
|{2025-03-09 16:53:30, 2025-03-09 16:53:40}|10   |
|{2025-03-09 16:53:35, 2025-03-09 16:53:45}|10   |
|{2025-03-09 16:53:40, 2025-03-09 16:53:50}|10   |
|{2025-03-09 16:53:45, 2025-03-09 16:53:55}|10   |
|{2025-03-09 16:53:50, 2025-03-09 16:54:00}|10   |
```

----------X----------